

Comp 370 Mini-Project 1 Final Report

1. Information on team members

- a. Jakob Upton | jakob.upton@student.ufv.ca | 300146320
- b. Gavin McNaughton | gavin.mcnaughton@student.ufv.ca | 300218601
- c. Jakob Hobek | jakob.hobek@student.ufv.ca | 300201635
- d. Donald Okonkwo | donald.okonkwo@student.ufv.ca | 300226980
- e. Massimo Currier | Massimo.currier@student.ufv.ca | 300193371

2. Summary of team contributions

- a. Jakob U: Repo management, Server/Monitor Implementation, Testing
- b. Gavin M: Task breakdown/assignment/timeline, Requirements, Report
- c. Jakob H: Client Implementation, Scripts
- d. Donald O: Logging, UML
- e. Massimo: Failover logic, Election algorithm

3. Requirements

- a. Stakeholders
 - i. System users (clients)
 - ii. System admin
 - iii. Development team
 - iv. Instructor
 - v. Monitor component
 - vi. Backup servers
- b. Problem statement: This project is centered around the problem of maintaining service availability in a distributed system in the case of primary server failure. It implements a Primary-Backup architecture with a heartbeat-centric failure detection protocol, ensuring automatic failover should the primary ever go offline for any reason.

	Functional Requirements	Non-Functional Requirements
1	The system shall support at least three independent server processes, each capable of listening on a unique port, that can be started/stopped independently.	The system shall detect “Primary” failure in 5 missed heartbeats or less.
2	The server shall maintain a Primary-Backup architecture wherein exactly one server is given the “Primary” designation and all remaining servers are “Backups”.	The system shall resume serving client requests within 2 seconds following failover election.

3	The system shall allow clients to send simple requests such as PING or PROCESS, receiving a valid response from the current “Primary”.	Under single “Primary” failure, the system should continue providing service with minimal disruption by quickly promoting a “Backup” and allowing client reconnection.
4	Each server shall periodically send heartbeat messages to the monitor and any other servers to signal liveness.	Following failover election, any new “Primary” shall serve requests using the latest replicated state in order to minimize inconsistencies.
5	The monitor shall track registered servers and detect any failures, signalled by missing heartbeats.	All key cluster events shall be observable through timestamped logs, and be sufficient to calculate detection time and failover time during testing.
6	Should the “Primary” fail, the monitor shall automatically PROMOTE a “Backup” to “Primary”.	The server shall handle multiple connections using concurrency without blocking the whole server on one client connection.
7	During failover election, the monitor shall select a new “Primary” based on the lowest server ID.	The server design shall follow OOP principles to ensure components are modular and testable.
8	The system shall utilize a split-brain avoidance approach such that two servers are not labeled “Primary” simultaneously.	The system shall support the given failure scenarios and make it feasible to repeat them consistently.
9	The client shall query the monitor to discover the current “Primary” host/port before sending service requests.	The system shall tolerate reasonable simulated heartbeat delays without triggering failover through tuning thresholds and documenting of results.
10	If the client’s connection to the “Primary” should fail, the client shall re-query the monitor and retry connecting until a successful connection is achieved.	The system shall run in accordance with typical Java programs, using sockets, and without platform-specific dependencies beyond scripts.
11	Monitor/Servers shall log key events with timestamps to console and/or files.	
12	The system shall provide a way to observe server status/health using	

	logs and/or live status output.	
13	The system shall provide an admin interface that can view recent logs/status, check server health, and trigger manual failover.	
14	The system shall implement and describe a simple state replication approach such that any “Backup” can take over consistently.	

Table 1: Functional and Non-Functional Requirements

4. Project objectives and our solution

The main objectives of this project were to (1) implement a server architecture with primary and backup servers, (2) implement a monitor and client that are capable of interfacing with the servers and maintaining server uptime, and (3) provide a mechanism through which servers can be promoted to primary in the event that the current primary times out. Given that this was a team project, our first priority was to break the tasks down into manageable and assignable segments. The result of this can be viewed in docs/ProjectBreakdown.pdf, essentially boiling down to a timeline of when things should be done, who does what, and what each task entails. After everything was broken down and assigned, we got to work implementing each segment. The main segments are the servers, the client, the admin console, and the monitor. These segments talk to each other, and maintain equilibrium through routine checks based on heartbeat signals that each server sends to the monitor. In addition to this, everything is logged and timestamped so troubleshooting is not a problem. As a final layer, we added the required tests to simulate failure in several ways as a demonstration of how our architecture handles problems.

5. Functional overview of main classes

- a. **SrmsNode**: Provides shared utilities for the monitor and server, including lifecycle control, parsing args, sleeping, and consistent logging.
- b. **LogWriter**: Enables thread-safe logging for each component.
- c. **ServerRole**: Outlines the current role of a server, either PRIMARY or BACKUP.
- d. **MessageSerializer**: Defines how messages between the monitor, server, and client will be serialized and deserialized.
- e. **MessageSocket**: Acts as a smaller wrapper for a TCP socket that sends and receives messages and automatically parses them with MessageSerializer.
- f. **ServerMonitor**: Accepts heartbeats, tracks all connected servers, assigns server IDs and roles, detects PRIMARY disconnection and promotes backup, responds to all GETPRIMARY queries.

- g. **ServerProcess:** Acts as the instantiation of a single server. All instances are capable of opening ports for a client, connecting to the monitor, sending heartbeats, and responding to PROCESS requests from a client.
- h. **Client:** Maintains a live connection to the PRIMARY server, repeatedly issues PROCESS requests, switches to the new PRIMARY should the old one fail for any reason.
- i. **AdminConsole:** Reads the monitor's log file in order to generate a server health summary. Also capable of printing recent logs for further examination.

6. Overview of state replication protocol & election algorithm

In our implementation, the "state" being maintained by the program includes several features: server role, server ID, and the monitor's record of current primary/backups and heartbeats. When a server starts up, it connects to the monitor and is given a serverID and currentRole. Each server repeatedly sends heartbeat messages to the monitor, which replies with an ACK after updating the current state. Should the current PRIMARY disconnect for any reason, the monitor removes it from the server list before electing a new primary using lowest numeric serverID.

7. Test results & screenshots (screenshots in docs/ folder)

- a. **after_heartbeat_delay:** This screenshot shows the output of the monitor and server processes following a 3 second delay in network traffic.
- b. **after_primary_kill:** This screenshot shows the outputs after the primary (s1) is killed. Note the connection attempt from the client, logs showing the disconnect and promotion in the monitor, and loss of heartbeat logs from S1 in the servers.
- c. **client_lost_connection:** This screenshot shows the output of the client when it's unable to connect to either the primary or the monitor. It attempts to connect 5 times before exiting.
- d. **server_monitor_client:** This screenshot shows typical output when all three components are running without issue.
- e. **server_monitor_noclient:** This screenshot shows typical output when only the monitor and servers are running.

8. Lessons learned & future improvements

While the overall progress on this project went quite smoothly, there were definitely several lessons learned and things that could be improved upon. Firstly, we figured out quite early on that failover is easy to demo but hard to actually implement correctly. Our strategy of promoting the lowest serverID can experience problems during certain edge cases like partial failures or delayed messages. Additionally, heartbeats are not sufficient to act as failure detection. There may be cases wherein a server is alive but unreachable/delayed, so missed heartbeats would cause false failovers. If we had more time/experience, it would be nice to revisit this and make some improvements to cover all possibilities. Outside of this, our design used centralized

coordination (having everything go through the monitor), which simplifies things but also adds one massive point of failure. If anything happens to the monitor, things could go awry quite quickly. During testing the monitor never failed to do its job, but under higher loads this could become a problem. One thing we found very helpful was to simplify our message protocols. Having small messages kept everything manageable, while simultaneously highlighting the importance of validation and error handling.